

Day 7: SQL, NoSQL and Data Storage Concepts for AI Systems

1. 5-line beginner summary

Databases are where AI systems store raw data, processed data, documents, chunks, embeddings, user queries, and model responses. SQL databases store structured data in tables like Excel sheets. NoSQL databases store flexible data such as JSON documents, key-value data, logs, sessions, and chat history. In GenAI and RAG systems, documents are split into chunks, converted into embeddings, and stored with metadata. A good AI system needs the right database design so retrieval, tracking, monitoring, and governance work properly.

2. Why databases matter in AI projects

AI models do not work alone. In real enterprise projects, AI systems need data from many sources.

Examples:

- Customer records from SQL databases
- Policy documents from document stores
- Application logs from NoSQL systems
- Embeddings from vector databases
- User queries and model responses for monitoring
- Metadata for access control, document source, date, department, version

A model gives better output when the data pipeline is clean, searchable, secure, and well-organized.

In simple words:

Bad data storage -> poor retrieval -> weak AI answer
Good data storage -> better retrieval -> better AI answer

3. SQL basics

SQL means **Structured Query Language**.

It is used to store and query structured data.

Structured data means data that fits neatly into rows and columns.

Example:

employee_id	name	department	salary
101	Ravi	AI	90000
102	Priya	Data	85000
103	Aman	Cloud	80000

SQL databases are commonly used for:

- Employee records
- Banking transactions
- Customer information
- Orders
- Invoices
- Product catalogs
- Audit data

Common SQL databases:

Database	Common use
PostgreSQL	Enterprise apps, analytics
MySQL	Web applications
SQL Server	Microsoft enterprise systems
Oracle	Large enterprise systems
DB2	IBM enterprise systems

4. Tables, rows and columns

A SQL database stores data in **tables**.

Think of a table like an Excel sheet.

Table: customers			
+-----+	+-----+	+-----+	+-----+
customer_id	name	city	
+-----+	+-----+	+-----+	+-----+
1	Ravi	Delhi	
2	Priya	Bengaluru	
3	Aman	Pune	
+-----+	+-----+	+-----+	+-----+

Important terms:

Term	Meaning
Table	Collection of related data
Row	One record
Column	One attribute or field
Primary key	Unique ID for each row
Foreign key	A reference to another table

Example:

customer_id is a primary key because it uniquely identifies each customer.

5. Easy SQL examples

Example 1: Select all data

```
SELECT *  
FROM customers;
```

Meaning:

Show me all customer records.

Example 2: Select specific columns

```
SELECT name, city  
FROM customers;
```

Meaning:

Show only name and city.

Example 3: Filter data

```
SELECT *  
FROM customers  
WHERE city = 'Delhi';
```

Meaning:

Show customers who live in Delhi.

Example 4: Sort data

```
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

Meaning:

Show employees from highest salary to lowest salary.

Example 5: Limit records

```
SELECT *  
FROM employees  
LIMIT 5;
```

Meaning:

Show only first 5 records.

6. Joins

A **join** combines data from multiple tables.

Example:

Table: customers

```
+-----+-----+
| customer_id | name  |
+-----+-----+
| 1           | Ravi  |
| 2           | Priya |
+-----+-----+
```

Table: orders

```
+-----+-----+-----+
| order_id | customer_id | amount |
+-----+-----+-----+
| 501      | 1           | 1000   |
| 502      | 2           | 1500   |
+-----+-----+-----+
```

To get customer name with order amount:

```
SELECT customers.name, orders.amount
FROM customers
JOIN orders
ON customers.customer_id = orders.customer_id;
```

Output:

```
+-----+-----+
| name  | amount |
+-----+-----+
| Ravi  | 1000   |
| Priya | 1500   |
+-----+-----+
```

Common join types

Join type	Meaning
INNER JOIN	Matching records from both tables
LEFT JOIN	All records from left table and matching records from right table

Join type	Meaning
RIGHT JOIN	All records from right table and matching records from left table
FULL JOIN	All records from both tables

For interviews, focus mainly on **INNER JOIN** and **LEFT JOIN**.

7. Aggregations

Aggregation means summarizing data.

Common aggregation functions:

Function	Meaning
COUNT()	Count rows
SUM()	Add values
AVG()	Average
MIN()	Minimum value
MAX()	Maximum value

Example: Count employees by department

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

Meaning:

```
Group employees by department and count how many employees are in each
department.
```

Example: Average salary by department

```
SELECT department, AVG(salary) AS average_salary
FROM employees
GROUP BY department;
```

Example: Total order amount by customer

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

Example: Filter after aggregation

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 5000;
```

Important difference:

Clause	Used for
WHERE	Filters rows before grouping
HAVING	Filters groups after aggregation

8. NoSQL basics

NoSQL means **Not Only SQL**.

NoSQL databases are used when data is flexible, large, fast-changing, or semi-structured.

SQL is like an Excel sheet.

NoSQL can be like:

- JSON file
- Dictionary
- Document folder
- Log storage
- Cache storage
- Graph structure

Common NoSQL databases:

Type	Example databases	Common use
Document database	MongoDB, Couchbase	JSON documents, user profiles
Key-value store	Redis, DynamoDB	Cache, sessions, fast lookup
Wide-column store	Cassandra, HBase	Large-scale event data
Graph database	Neo4j	Relationships, network data

9. Document databases

Document databases store data as JSON-like documents.

Example user profile:

```
{
  "user_id": "U101",
  "name": "Ravi",
  "city": "Delhi",
  "skills": ["Python", "SQL", "GenAI"],
```

```
"preferences": {  
  "language": "English",  
  "theme": "dark"  
}  
}
```

This is useful because the structure can be flexible.

One user may have skills. Another user may have preferences. Another user may have address details.

Document databases are useful for AI systems because they can store:

- Chat history
- User profile
- Uploaded document metadata
- JSON responses
- Model configuration
- Prompt templates
- Feedback records

10. Key-value stores

A key-value store works like a dictionary.

Example:

```
Key: user:101:session  
Value: active
```

Another example:

```
Key: query:abc123  
Value: "What is my claim status?"
```

Common key-value database:

```
Redis
```

Useful for:

- Caching
- Session storage
- Temporary data
- Fast lookups
- Rate limiting
- Conversation state

Example in AI system:

User asks same question again.
Instead of calling LLM again, check cache.
If answer exists, return cached answer.

11. SQL vs NoSQL

Area	SQL	NoSQL
Data structure	Fixed tables	Flexible structure
Schema	Strict schema	Flexible schema
Query language	SQL	Depends on database
Best for	Structured business data	Flexible, large, semi-structured data
Example data	Employees, orders, transactions	JSON documents, logs, chat history
Scaling	Usually vertical + some horizontal	Usually horizontal
Consistency	Strong consistency common	Depends on database
Example DB	PostgreSQL, MySQL, DB2	MongoDB, Redis, Cassandra

Simple rule:

Use SQL when data is structured and relationships matter.
Use NoSQL when data is flexible, large, or changes frequently.

For AI systems, both are often used together.

12. Storing documents

In GenAI systems, users may upload documents like:

- PDF files
- Word files
- Policy documents
- HR documents
- Product manuals
- Support tickets
- Knowledge base articles

Usually, the full document is stored in:

- Object storage, such as cloud storage
- Document database
- File system
- Data lake

Example document metadata table:

documents			
+-----+-----+-----+-----+			
document_id	file_name	department	upload_date
+-----+-----+-----+-----+			
D101	hr_policy.pdf	HR	2026-07-01
D102	cloud_guide.pdf	IT	2026-07-02
+-----+-----+-----+-----+			

The actual PDF may be stored in object storage, while metadata is stored in SQL or NoSQL.

13. Storing chunks

Large documents are usually split into smaller pieces called **chunks**.

Why?

Because LLMs cannot process unlimited text at once. Also, retrieval works better when we search smaller sections instead of huge documents.

Example:

Original document:	
"IBM HR policy contains leave policy, work from home policy, travel policy..."	
Chunks:	
Chunk 1: Leave policy details	
Chunk 2: Work from home policy details	
Chunk 3: Travel policy details	

Example chunk table:

chunks			
+-----+-----+-----+-----+			
chunk_id	document_id	chunk_number	chunk_text
+-----+-----+-----+-----+			
C001	D101	1	Leave policy details...
C002	D101	2	Work from home policy...
C003	D101	3	Travel policy details...
+-----+-----+-----+-----+			

Important chunk fields:

Field	Purpose
chunk_id	Unique chunk ID
document_id	Link to original document
chunk_text	Actual text
chunk_number	Position in document

Field	Purpose
page_number	Source page
section_title	Heading
created_at	When chunk was created

14. Storing embeddings

An **embedding** is a numerical representation of text.

Example text:

```
"What is the leave policy?"
```

Embedding:

```
[0.12, -0.45, 0.87, 0.33, ...]
```

The numbers capture meaning.

Similar sentences have similar embeddings.

Example:

```
"What is the leave policy?"
"How many leaves can I take?"
```

These may have similar embeddings because the meaning is close.

Embeddings are usually stored in a **vector database** or a database with vector search support.

Examples:

- Milvus
- Pinecone
- Weaviate
- Chroma
- FAISS
- PostgreSQL with pgvector
- Elasticsearch/OpenSearch vector search

Example embedding storage:

```
chunk_embeddings

+-----+-----+-----+
| chunk_id | document_id | embedding          |
+-----+-----+-----+
| C001     | D101        | [0.12, -0.45, 0.87...] |
| C002     | D101        | [0.22, -0.11, 0.76...] |
+-----+-----+-----+
```

In real systems, the embedding vector can have hundreds or thousands of dimensions.

15. Storing user queries and model responses

Enterprise AI systems should store user questions and model responses for:

- Debugging
- Quality improvement
- Monitoring
- Auditing
- Feedback analysis
- Cost tracking
- Safety checks
- Compliance

Example table:

```
ai_interactions
+-----+-----+-----+
+-----+
| interaction_id | user_id | user_query          | model_response
|
+-----+-----+-----+
+-----+
| I001          | U101    | What is leave policy? | Employees get...
|
| I002          | U102    | Explain travel policy  | Travel policy says...
|
+-----+-----+-----+
+-----+
```

Additional useful fields:

Field	Purpose
model_name	Which model was used
prompt_version	Which prompt template was used
retrieved_chunk_ids	Which chunks were used
latency_ms	Response time
input_tokens	Prompt token count
output_tokens	Response token count
cost	Model call cost
user_feedback	Like/dislike/rating
created_at	Time of interaction

This is very important in enterprise AI because teams need to know:

What did the user ask?
What did the AI answer?
Which data source was used?
Was the answer correct?
How much did it cost?

16. Metadata storage for AI applications

Metadata means **data about data**.

Example:

Actual data:

Leave policy text

Metadata:

Document name: HR Policy 2026
Department: HR
Page number: 12
Confidentiality: Internal
Uploaded by: admin
Version: v3
Created date: 2026-07-01

Metadata helps AI systems retrieve the right information.

Example:

User asks:

Show me only HR policy documents updated after 2025.

The system can filter using metadata.

Example metadata table:

```
document_metadata

+-----+-----+-----+-----+-----+
| document_id | department | version | access  | effective_date |
+-----+-----+-----+-----+-----+
| D101        | HR         | v3      | internal | 2026-01-01     |
| D102        | Finance    | v2      | restricted | 2025-08-01     |
+-----+-----+-----+-----+-----+
```

In RAG systems, metadata is extremely important because vector search alone is not enough.

Example:

Without metadata:
The system may retrieve old HR policy.

With metadata:

The system retrieves latest HR policy, version v3, department HR.

17. How databases connect with AI pipelines

A typical AI pipeline looks like this:

```
Raw Data Sources
|
v
Data Ingestion
|
v
Data Cleaning / Preprocessing
|
v
Storage Layer
|
v
Feature Engineering / Chunking / Embedding
|
v
Model Training or RAG Retrieval
|
v
Model Output
|
v
Monitoring, Feedback and Audit Storage
```

In traditional ML:

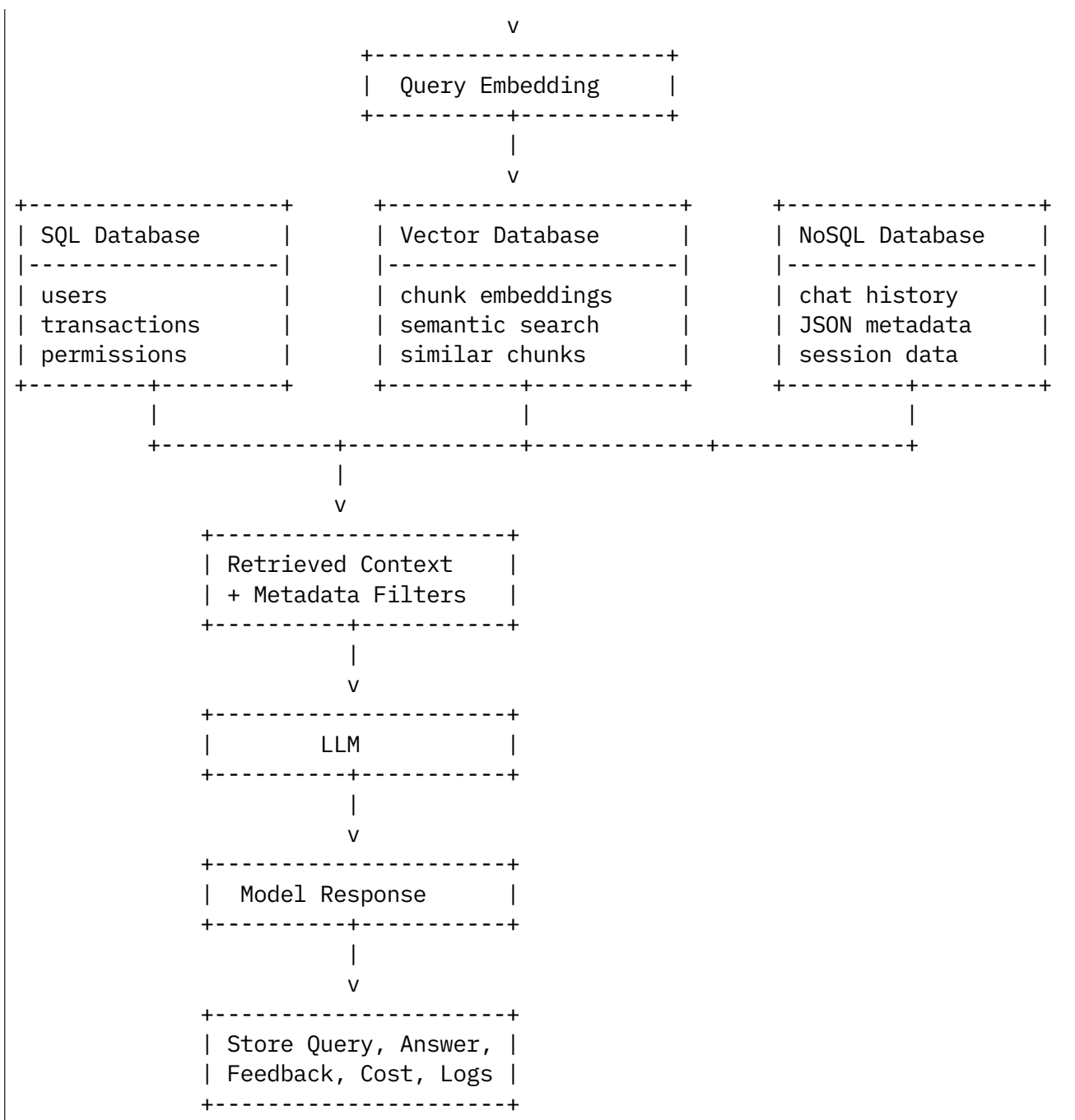
```
Database -> clean data -> features -> train model -> predictions -> store
predictions
```

In GenAI/RAG:

```
Documents -> chunks -> embeddings -> vector DB -> retrieve context -> LLM answer
```

18. ASCII diagram showing database usage in AI system

```
+-----+
| User / Application |
+-----+
|
v
+-----+
| User Query         |
+-----+
|
```



19. SQL example for AI application data

Users table

```

CREATE TABLE users (
  user_id INT PRIMARY KEY,
  name VARCHAR(100),
  department VARCHAR(100),
  role VARCHAR(100)
);

```

Documents table

```
CREATE TABLE documents (  
  document_id VARCHAR(50) PRIMARY KEY,  
  file_name VARCHAR(255),  
  department VARCHAR(100),  
  version VARCHAR(20),  
  uploaded_at TIMESTAMP  
);
```

Chunks table

```
CREATE TABLE chunks (  
  chunk_id VARCHAR(50) PRIMARY KEY,  
  document_id VARCHAR(50),  
  chunk_text TEXT,  
  page_number INT,  
  section_title VARCHAR(255),  
  FOREIGN KEY (document_id) REFERENCES documents(document_id)  
);
```

AI interactions table

```
CREATE TABLE ai_interactions (  
  interaction_id VARCHAR(50) PRIMARY KEY,  
  user_id INT,  
  user_query TEXT,  
  model_response TEXT,  
  model_name VARCHAR(100),  
  input_tokens INT,  
  output_tokens INT,  
  latency_ms INT,  
  created_at TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

20. Query examples for AI system

Find all documents from HR department

```
SELECT *  
FROM documents  
WHERE department = 'HR';
```

Find chunks for one document

```
SELECT chunk_id, chunk_text, page_number  
FROM chunks  
WHERE document_id = 'D101'  
ORDER BY page_number;
```

Count AI interactions per user

```
SELECT user_id, COUNT(*) AS total_queries
FROM ai_interactions
GROUP BY user_id;
```

Find slow AI responses

```
SELECT interaction_id, user_query, latency_ms
FROM ai_interactions
WHERE latency_ms > 5000;
```

Find most used model

```
SELECT model_name, COUNT(*) AS usage_count
FROM ai_interactions
GROUP BY model_name
ORDER BY usage_count DESC;
```

21. Pseudocode for storing and retrieving AI application data

A. Store document, chunks and embeddings

```
FUNCTION ingest_document(file):

    document_id = generate_unique_id()

    extracted_text = extract_text_from_file(file)

    store document metadata:
        document_id
        file_name
        department
        upload_date
        version

    chunks = split_text_into_chunks(extracted_text)

    FOR each chunk in chunks:

        chunk_id = generate_unique_id()

        store chunk:
            chunk_id
            document_id
            chunk_text
            page_number
            section_title

        embedding = create_embedding(chunk_text)
```



```
store embedding in vector database:
    chunk_id
    document_id
    embedding
    metadata

RETURN "Document ingested successfully"
```

B. Retrieve answer using RAG

```
FUNCTION answer_user_question(user_id, question):

    interaction_id = generate_unique_id()

    user_info = get_user_from_sql_database(user_id)

    question_embedding = create_embedding(question)

    similar_chunks = search_vector_database(
        embedding = question_embedding,
        filters = {
            department: user_info.department,
            access_level: user_info.role
        },
        top_k = 5
    )

    context = combine_chunk_texts(similar_chunks)

    prompt = create_prompt(
        question = question,
        context = context
    )

    response = call_llm(prompt)

    store interaction:
        interaction_id
        user_id
        question
        response
        retrieved_chunk_ids
        model_name
        token_count
        latency
        timestamp

    RETURN response
```

C. Store user feedback

```
FUNCTION store_feedback(interaction_id, rating, comment):  
  
    update ai_interactions table:  
        set user_rating = rating  
        set user_comment = comment  
        where interaction_id = interaction_id  
  
    RETURN "Feedback saved"
```

D. Use cache for repeated questions

```
FUNCTION answer_with_cache(user_id, question):  
  
    cache_key = create_hash(user_id, question)  
  
    cached_answer = get_from_key_value_store(cache_key)  
  
    IF cached_answer exists:  
        RETURN cached_answer  
  
    answer = answer_user_question(user_id, question)  
  
    save_to_key_value_store(  
        key = cache_key,  
        value = answer,  
        expiry = 1 hour  
    )  
  
    RETURN answer
```

22. How SQL, NoSQL and vector databases work together

In a real GenAI application, you may use multiple databases.

```
SQL Database:  
- Users  
- Roles  
- Permissions  
- Transactions  
- Audit records  
  
NoSQL Database:  
- Chat history  
- JSON metadata  
- Session data  
- Application logs  
  
Vector Database:  
- Embeddings  
- Semantic search
```

- Similar chunk retrieval

Object Storage:

- Original PDFs
- Word documents
- Images
- Audio files

Example:

User asks:

"What is the reimbursement policy for my department?"

System does this:

1. Check user department from SQL database.
2. Convert question into embedding.
3. Search vector database for matching reimbursement chunks.
4. Filter chunks using metadata.
5. Send selected chunks to LLM.
6. Store query and response in SQL/NoSQL.
7. Save feedback and monitoring logs.

23. Practical enterprise example

Imagine an internal IBM-style HR assistant.

User asks:

How many days of annual leave do I get?

Behind the scenes:

1. User profile is fetched from SQL database.
2. HR policy PDF is already stored in object storage.
3. HR policy text was split into chunks.
4. Each chunk has an embedding stored in vector database.
5. The question is converted into an embedding.
6. Similar chunks are retrieved.
7. Metadata filter checks latest policy version.
8. LLM generates answer using retrieved policy text.
9. Query, response, chunks used, model name and feedback are stored.

This is why databases are core to AI systems.

24. Common mistakes

Mistake 1: Storing everything in one database

Bad approach:

Store users, PDFs, embeddings, logs, and sessions in one table.

Better approach:

Use SQL for structured records.
Use object storage for files.
Use vector DB for embeddings.
Use NoSQL for flexible logs and chat history.

Mistake 2: Not storing metadata

Without metadata, retrieval becomes weak.

Bad:

Store only chunk_text and embedding.

Better:

Store chunk_text, document_id, page_number, department, version, access level, date.

Mistake 3: Poor chunking

Bad chunk:

Huge 20-page text block

Problem:

Retrieval becomes noisy.

Better:

Smaller meaningful chunks based on headings, paragraphs, and sections.

Mistake 4: Not tracking user queries and model responses

Bad:

Only show answer to user and forget everything.

Problem:

No debugging, no monitoring, no improvement.

Better:

Store user query, response, retrieved chunks, prompt version, model name, latency, cost, feedback.

Mistake 5: Ignoring access control

Bad:

```
Retrieve all documents for every user.
```

Problem:

```
User may see confidential information.
```

Better:

```
Use metadata filters based on role, department and permission.
```

Mistake 6: Confusing SQL search with semantic search

SQL search:

```
Find exact or pattern match.
```

Vector search:

```
Find meaning-based match.
```

Example:

User asks:

```
How many vacation days do I get?
```

Document says:

```
Annual leave entitlement is 20 days.
```

Keyword search may miss it. Vector search may find it because the meaning is similar.

Mistake 7: Not versioning documents

Bad:

```
Use any matching HR policy.
```

Better:

```
Use only latest approved HR policy.
```

Metadata should include:

```
version
effective_date
expiry_date
approval_status
```

25. Simple interview-ready explanation

For AI systems, databases are not just for storing records. They support the complete AI lifecycle. SQL databases store structured business data such as users, permissions and transactions. NoSQL databases store flexible data such as JSON documents, logs, sessions and chat history. In GenAI and RAG systems, documents are stored, split into chunks, converted into embeddings, and indexed in a vector database. Metadata such as source, version, department and access level helps retrieve the right information. The AI pipeline connects all of these storage layers to generate accurate, secure and traceable responses.

26. Day 7 revision checklist

You should be comfortable explaining:

- [] What SQL is
- [] What NoSQL is
- [] Difference between table, row and column
- [] Primary key and foreign key
- [] INNER JOIN and LEFT JOIN
- [] GROUP BY and aggregation
- [] SQL vs NoSQL
- [] Document database use case
- [] Key-value store use case
- [] Why documents are split into chunks
- [] What embeddings are
- [] Why vector databases are used
- [] Why metadata matters in RAG
- [] How user queries and model responses are stored
- [] How databases connect with AI pipelines

27. One-line memory hook

SQL stores structured business data, NoSQL stores flexible application data, and vector databases store embeddings for meaning-based search in AI systems.